

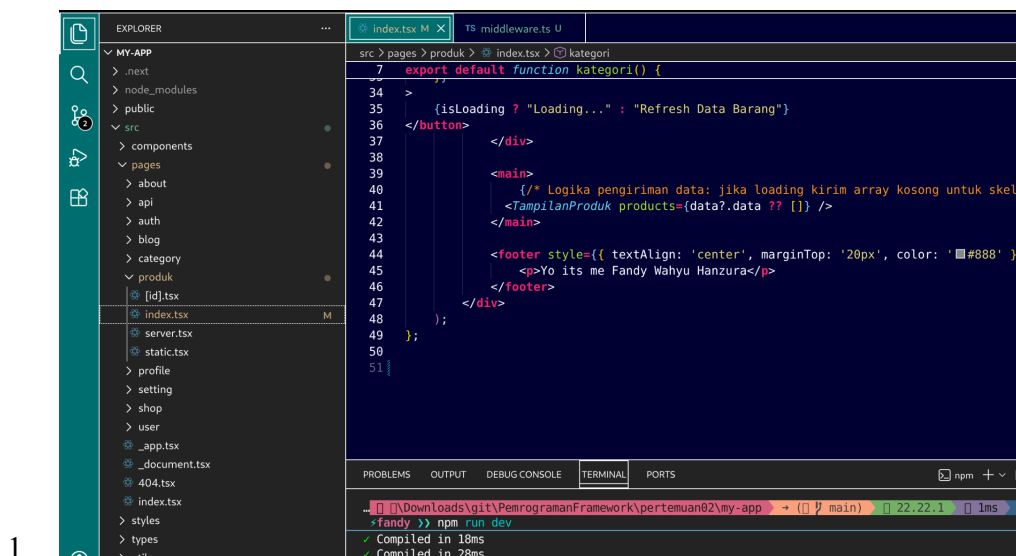


Mata Kuliah : Pemrograman Framework
Program Studi : D4 – Teknik Informatika
Semester : 6

Kelas : TI-3C
NIM : 2341720165
Nama : Fandy Wahyu Hanzura
Jobsheet Ke- : 12

Laporan Jobsheet

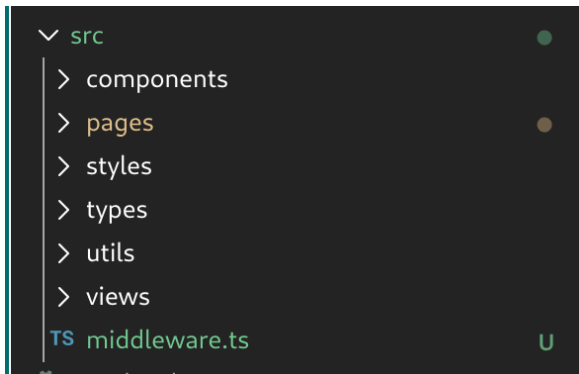
Praktikum 1



- Melakukan editing pada file index agar bisa menggunakan routing



2.

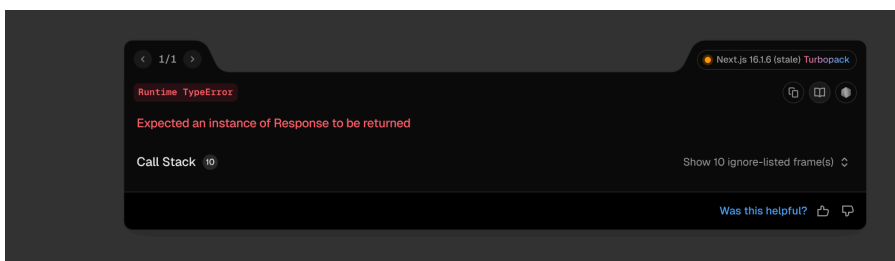


```
src > TS middleware.ts > middleware
1 import { NextRequest, NextResponse } from "next/server";
2
3 export function middleware(request: NextRequest) {
4   return NextResponse.next();
5 }
```

- menambahkan fungsi middleware pada file middleware

3.

```
src > TS middleware.ts > middleware
1 import { NextRequest, NextResponse } from "next/server";
2
3 export function middleware(request: NextRequest) {
4   return NextResponse.redirect(new URL("/", request.url));
5   // return NextResponse.next();
6 }
```

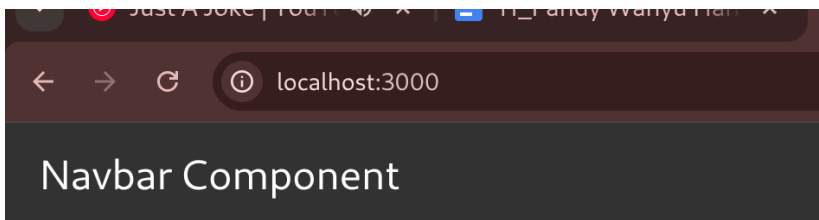


- Mencoba melakukan redirect pada halaman / sehingga terbitlah error



```
src > TS middleware.ts > [E] config
1  import { NextRequest, NextResponse } from "next/server";
2
3  export function middleware(request: NextRequest) {
4      return NextResponse.redirect(new URL("/", request.url));
5      // return NextResponse.next();
6  }
7
8  export const config = {
9      matcher: ["/produk", "/about"],
10 }
```

4.

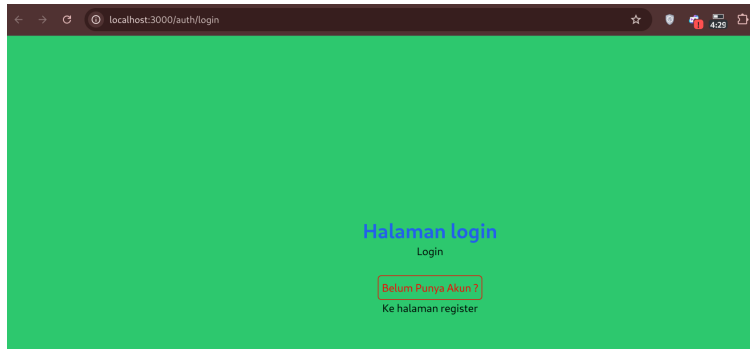


Praktikum Next.js Pages Router
Mahasiswa D4 Pengembangan Web

- Menambahkan exception pada file middleware

```
src > TS middleware.ts > ...
1  import { NextRequest, NextResponse } from "next/server";
2
3  export function middleware(request: NextRequest) {
4      const isLogin = false;
5      if(!isLogin) {
6          return NextResponse.next();
7      } else{
8          return NextResponse.redirect(new URL("/auth/login", request.url));
9      }
10     // return NextResponse.redirect(new URL("/", request.url));
11     // return NextResponse.next();
12 }
13
14 export const config = {
15     matcher: ["/produk", "/about"],
16 }
```

5.



- Menambahkan kondisi jika belum login akan diarahkan kepada halaman login ketika akan melakukan akses ke dalam halaman about

Pertanyaan refleksi

1. Mengapa middleware lebih aman dibanding useEffect?

- Middleware berjalan di sisi server (*Server-Side*) sebelum sebuah halaman dikirimkan ke browser pengguna. Hal ini memastikan validasi keamanan (seperti pengecekan token atau sesi) dilakukan sebelum kode halaman apa pun dieksekusi oleh klien. Sebaliknya, `useEffect` berjalan di sisi klien (*Client-Side*), yang berarti kode halaman dan komponen sudah sempat diunduh oleh browser sebelum pemeriksaan keamanan dilakukan. Pihak yang tidak bertanggung jawab dapat dengan mudah memanipulasi atau menonaktifkan JavaScript di browser untuk melewati proteksi yang hanya mengandalkan `useEffect`.

2. Mengapa middleware tidak menimbulkan glitch?

- *Glitch* atau *flickering* (tampilan konten sekilas sebelum diarahkan/redirect) sering terjadi pada penggunaan `useEffect` karena halaman sempat dirender oleh browser sebelum fungsi pengalihan dijalankan. Middleware mencegah hal ini karena proses penentuan akses dilakukan pada *Request-Response cycle* di server. Jika pengguna tidak memiliki akses, server akan langsung memberikan respons *redirect* tanpa pernah mengirimkan konten halaman tujuan ke browser, sehingga transisi perpindahan halaman menjadi bersih tanpa gangguan visual.



3. Apa risiko jika semua halaman diproteksi tanpa pengecualian?

Jika seluruh rute diproteksi tanpa pengecualian (*whitelist*), maka aset-aset publik dan halaman krusial juga akan terkunci. Risikonya meliputi:

- **Halaman Login Tidak Dapat Diakses:** Pengguna tidak akan pernah bisa masuk ke aplikasi karena halaman Login itu sendiri terproteksi oleh aturan yang mengharuskan pengguna untuk login terlebih dahulu (*infinite redirect loop*).
- **Aset Statis Terhambat:** File gambar, CSS, dan JavaScript di folder `public` atau `_next` mungkin tidak akan termuat, yang mengakibatkan tampilan situs menjadi berantakan.
- **Penurunan Performa:** Middleware akan memproses setiap permintaan aset kecil, yang meningkatkan beban komputasi server secara tidak perlu.

4. Kapan middleware tidak diperlukan?

Middleware tidak diperlukan pada kondisi-kondisi berikut:

- **Konten Publik:** Halaman-halaman yang memang ditujukan untuk konsumsi umum tanpa memerlukan autentikasi (seperti Landing Page atau Blog publik).
- **Validasi Data Internal:** Untuk logika bisnis yang sangat kompleks dan membutuhkan akses database yang intensif, lebih disarankan menggunakan **API Routes** atau **Server Actions** agar tidak menghambat kecepatan respons awal (*Initial Response Time*).
- **Interaksi Sederhana UI:** Perubahan tampilan yang hanya bersifat kosmetik atau interaksi antarmuka yang tidak memerlukan validasi keamanan dari sisi server.

5. Apa perbedaan middleware dan API route?

Meskipun keduanya berjalan di server, perbedaan utamanya terletak pada **tujuan** dan **waktu eksekusi**:



- **Middleware:** Berfungsi sebagai "penjaga gerbang" (*Gatekeeper*) yang memproses permintaan **sebelum** mencapai rute tujuan (halaman atau API). Middleware digunakan untuk logika global seperti autentikasi, bot-detection, atau pengalihan bahasa.
- **API Route:** Berfungsi sebagai "penyedia data" (*Data Provider*) atau *endpoint* yang dipanggil secara eksplisit untuk menjalankan tugas tertentu, seperti menyimpan data ke database (CRUD) atau memproses transaksi. API Route hanya berjalan jika dipanggil, sedangkan Middleware berjalan pada setiap permintaan rute yang sesuai dengan *matcher*.